Comparing Machine Learning Models for Predicting Poverty Rates in World Bank Data

Claire Skatrud

Washington University in St. Louis, IES Abroad, Universidad Complutense de Madrid

December 2, 2025

Abstract

This project compares several machine learning models and their variations for predicting global poverty rates. The data for this project is the World Bank's Global Data of World Development Indicators, accessed from Kaggle. This paper describes the process of target indicator selection, feature selection, data preparation, feature importance, and the performances of models. I explore random forest, multivariable linear regression, and neural network models. For each model, performance was compared across subsequent adjustments like hyperparameter tuning, regularization, reducing correlated features, and adjusting model architecture. Maintaining explainability and interpretability was a priority, especially given the policy relevance of poverty measurement and model-driven insights. Ultimately, this work evaluates both the predictive ability and practical interpretability of different modeling approaches for an inherently complex global development problem.

Contents

1. Data
    1.1 Overview
        1.1.1 Dataset Structure
    1.2 Selecting a Target Indicator
    1.3 Selecting a Year Cross Section
2. Feature Selection
    2.1 Initial Approach
    2.2 Manual Selection of Topics
    2.3 Dropping by Threshold
    2.4 Reducing Highly Correlated Features
        2.4.1 Motivation
        2.4.2 Feature Reduction Methodology
3. Data Preparation
    3.1 Overall
    3.2 Merging, Slicing and Pivoting
    3.3 One-Hot Encoding Categorical Variables
    3.4 Assessing Null Values and Dropping Rows With High Missingness
    3.5 Imputing Remaining Missing Values
        3.5.1 Initial Approach: Imputing Based on Nearby Years
        3.5.2 Final Approach: Imputing by Median Values for Region Group
    3.6 Winsorizing the Data
    3.7 Scaling the Data
4. Random Forest Regression
    4.1 Complete Feature Set
        4.1.1 Random Forest Baseline Model
        4.1.2 Random Forest Tuned Model
        4.1.3 Feature Importance
    4.2 Reduced Feature Set
        4.2.1 Random Forest Baseline Model
        4.2.2 Random Forest Tuned Model
        4.2.3 Feature Importance
5. Multivariable Linear Regression
    5.1 Complete Feature Set
        5.1.1 Linear Regression Baseline Model
        5.1.2 Ridge Regression Model
        5.1.3 Ridge Regression Tuned Model
    5.2 Reduced Feature Set

5.2.1 Baseline Linear Regression Model (Reduced Features)
6. Neural Network
6.1 Complete Feature Set
6.1.1 Baseline Neural Network
6.1.2 Large Neural Network (Purposefully Overfitted)
6.1.3 Regularized Neural Network
6.1.4 Tuned Regularized Neural Network
6.1.5 Other Adjustments
6.2 Reduced Feature Set
6.2.1 Regularized Neural Network (Reduced Features)
7. Model Comparison
8. Conclusion
9. Appendix
9.1 Columns of Each CSV File
9.2 Selected Topics
9.3 Model Performance and Run Time
10. References

1. Data

1.1 Overview

The dataset for this project is called World Bank's Global Data on Kaggle and it consists of many World Development Indicators across economic, social, and environmental focuses. The data includes 1453 indicators for 265 countries from 1960 to 2023.

1.1.1 Dataset Structure

According to the Kaggle data card, the dataset is structured in the following CSV files:

- *series:* "Contains metadata for various indicators, including their descriptions, definitions, and other relevant information. This file acts as a reference for understanding what each indicator represents."
- *country_series:* "Establishes relationships between countries and specific indicators. It provides additional metadata, such as contextual descriptions of indicator usage for particular countries."
- *countries:* "Includes detailed information about countries, such as country codes, region classifications, income levels, and other geographical or socio-economic attributes."
- *footnotes:* "Provides supplementary notes and additional context for specific data points in the main dataset. These notes clarify exceptions, limitations, or other special considerations for particular entries."
- *main_data:* "The core dataset containing the actual indicator values for countries across different years. This file forms the backbone of the dataset and is used for analysis."
- *series_time:* "Contains time-related metadata for indicators, such as their start and end years or periods of data availability."

See the appendix (9.1) for the columns of each CSV file. The most important components of the data were the 'indicators,' each of which belonged to one of 89 'topics'. For each indicator, there is an entry for each country for each year between 1960 and 2023. It is important to note that there were many null values for different combinations of years, indicators, and countries. For this project, only a fraction of the total data was used, which will be explained in the feature selection section.

1.2 Selecting a Target Indicator

The selected target for this project is 'Poverty headcount ratio at $3.65 a day (2017 PPP) (% of population)'. This indicator had less null values than the other poverty rate indicators, which included ['Poverty gap at $2.15 a day (2017 PPP) (%)',
'Poverty headcount ratio at $3.65 a day (2017 PPP) (% of population)',
'Poverty gap at $3.65 a day (2017 PPP) (%)',
'Multidimensional poverty headcount ratio (UNDP) (% of population)',
'Multidimensional poverty headcount ratio (World Bank) (% of population)',
'Poverty headcount ratio at national poverty lines (% of population)',

'Poverty headcount ratio at societal poverty line (% of population)',
'Poverty headcount ratio at $6.85 a day (2017 PPP) (% of population)',
'Poverty gap at $6.85 a day (2017 PPP) (%)'].
After calculating the number of missing values per year for each indicator, 'Poverty headcount ratio at $3.65 a day (2017 PPP) (% of population)' appeared to have the most complete data.

1.3 Selecting a Year Cross Section

After determining the target indicator, I had to select which year had the most complete data to decide which cross section I should use for the machine learning models. To limit bias, the indicator completeness for a certain year was calculated as an average of countries grouped by income group.

	2008	2009	2010	2011	2012	2013	2014	2015	2016	2017	2018
Income Group											
High income	0.447059	0.435294	0.458824	0.435294	0.447059	0.470588	0.458824	0.447059	0.458824	0.470588	0.470588
Low income	0.076923	0.230769	0.230769	0.153846	0.115385	0.076923	0.230769	0.115385	0.192308	0.000000	0.269231
Lower middle income	0.196078	0.215686	0.274510	0.235294	0.294118	0.156863	0.176471	0.313725	0.196078	0.215686	0.294118
Upper middle income	0.425926	0.444444	0.425926	0.462963	0.518519	0.462963	0.518519	0.518519	0.500000	0.500000	0.537037

Figure 1: Proportion of non-null values by income group per year for target indicator (truncated to 2008-2018, which were the most complete years)

It was important to avoid selecting a year that was highly unrepresentative for countries of certain income groups. For example, there appears to be no data in 2017 for our target indicator for low income countries. This would be problematic because our model would not be trained or tested on any low income countries and would therefore miss important predictive information about poverty rates for that group.

Based on the table above, 2018 was selected as the year that has a higher completeness both overall and by income group.

2. Feature Selection

2.1 Initial Approach

Initially, my goal was to select a year cross section by identifying which year had the highest number of highly complete features (e.g. more non-null country data points for a certain indicator for a certain year). However, it quickly became apparent that some manual and automatic feature-limiting approaches were necessary.

2.2 Manual Selection of Topics

As mentioned in Section 1.1.1, each ‘indicator’ belonged to one of 89 ‘topics.’ I decided to manually select topics (groups of indicators) that might be more related to poverty rates and thus would have a more interpretable connection to our target variable. For example, selected topics included:

- ‘Social Protection & Labor: Unemployment’
- ‘'Economic Policy & Debt: National accounts: US$ at current prices: Aggregate indicators'
- ‘Education: Outcomes’

Meanwhile, dropped topics included:

- ‘Environment: Biodiversity & protected areas’
- ‘Public Sector: Defense & arms trade’
- 'Environment: Freshwater'

This is not to say the dropped topics and their indicators definitively would not have explanatory power for poverty rates, but rather were less immediately related. See the appendix (9.2) for the full list of selected topics. The manual selection resulted in 32 topics, which encompassed 617 indicators (out of the initial 1453 indicators).

Even after limiting the number of features, 617 is still a very large number and was not conducive to certain checks, like manually observing a chart of indicator completeness by year by income group (like table 1, for the target indicator).

2.3 Dropping by Threshold

Once 2018 was selected as the desired cross section (Section 1.3), the selected indicators’ completeness could be calculated for only 2018, specifically for countries for which the target indicator was non-null. In other words, each indicator is evaluated based on the proportion of non-null values in 2018 for countries that had an available poverty rate.

Several automated thresholds were applied.

- Any indicator with < 0.85 completeness (non-null values / total) in 2018 was dropped.
- For each indicator, a regional completeness score was calculated (average per region). The indicator needed to have > 0.85 completeness in at least 3 regions to remain included.
- For each indicator, an income group completeness score was calculated (average per income group). An indicator is dropped if it has < 0.5 completeness for any income group.

This further reduced the number of indicators from 617 to 218. This group of 218 selected features, plus the one-hot encoded ‘Region’ and ‘Income Group’ features (discussed elsewhere), formed the full columns of the working dataset on which the machine learning models were trained.

2.4 Reducing Highly Correlated Features

2.4.1 Motivation

Redundant features typically reduce accuracy of machine learning models. Additionally, model explainability or feature importance can be enhanced when highly correlated features are reduced.

2.4.2 Feature Reduction Methodology

In addition to training the models on the 227 selected features, I also created a reduced feature set that grouped highly correlated features. I used the Networkx Python package to create a graph of features with weights representing the correlations. Then, it formed and printed 85 groups of highly correlated features. I manually selected one primary feature from each, seeking to preserve clarity and explainability. For example, in one group there are 25 features that are highly correlated, one of which is “GDP (current US$)”, so I selected that as the representative feature from that group. Many groups only included a few features, such as “Unemployment, female (% of female labor force)”, “Unemployment, male (% of male labor force)”, and “Unemployment, total (% of total labor force)”. In this case, I chose “Unemployment, total (% of total labor force)” to be the representative feature.

The result was a list of 85 features, which I refer to as the reduced feature set and used to test various models later on to compare performance with the complete feature set of 227.

3. Data Preparation

3.1 Overall

Preparing this data, aside from the feature selection and year cross section selection, involved several steps. Certain data tables had to be merged, sliced and pivoted to become workable in a machine learning format, and I one-hot encoded the categorical variables ‘Income Group’ and ‘Region’. Then, I addressed null values (by row/country, rather than by indicator as before), imputed missing values, and winsorized the data to reduce skewedness.

3.2 Merging, Slicing and Pivoting

Region and Income Group were both columns on the ‘countries’ table. Before proceeding, I left-merged those columns into the ‘main_data’ table so they could be included as features.

Once 2018 was selected as the cross section of interest and the indicators were selected (see Sections 1.4 and 2.2), I created a dataframe called working_data_2018. This was a slice of the 'main_data' dataframe that pivoted the data to make the indicators columns instead of rows, as they were originally in 'main_data'.

'main_data' table columns
'Country Name' 'Country Code' 'Indicator Name' 'Indicator Code' '1960' '1961' …. '2022' '2023'

'countries' table columns
'Country Code' 'Short Name' 'Region' 'Income Group' ….

Figure 2: Initial columns of the relevant tables

As shown above, the 'main_data' table originally had the years as columns, with countries and indicators filling the rows. After merging, slicing, and pivoting, this was the working data:

'working_data_2018' columns
'Country Name' 'Country Code' 'Region' 'Income Group' 'Indicator 1' 'Indicator 2' … 'Indicator 218' 'Target Variable'

Figure 3: Columns of the pivoted data ('Indicator 1' etc. serves as a stand-in here for the longer indicator name)

This leaves us with a table where each row is a country, and the columns are the indicators.

3.3 One-Hot Encoding Categorical Variables

'Region' and 'Group' were one-hot encoded to allow them to be included in the machine learning processes later on. I used the get_dummies method from Pandas, which creates a column for each label in the category and defines each entry as 0 or 1. All other features were numeric.

3.4 Assessing Null Values and Dropping Rows With High Missingness

Although null values were already in part addressed by our feature selection, I now had to look at null values on the row/country-level.

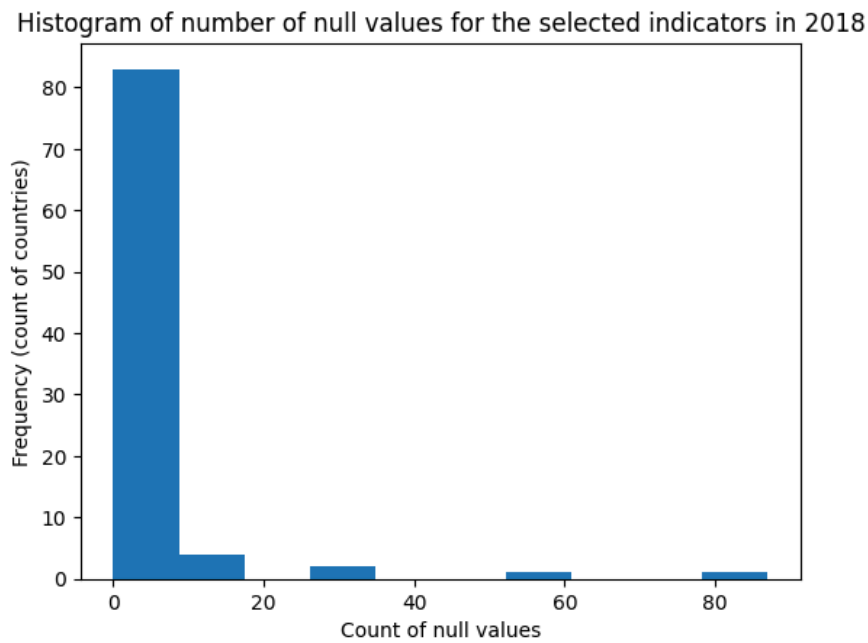


Figure 4: Histogram of the number of null values per indicator in the working data

To address that, I removed rows with too many null values. Any row with over 20 null values, which is about 22% of the total number of rows (91), was dropped.

3.5 Imputing Remaining Missing Values

For the remaining null values, data was imputed. Imputation is important to ensure machine learning models run correctly and do not get errors due to missing data.

3.5.1 Initial Approach: Imputing Based on Nearby Years

Since our dataset does include years before and after 2018, my first approach was to try to impute rows based on interpolation from nearby years. This would hopefully be more precise and relevant to the country than filling in an average value for all countries, especially because there is so much variation in many of these indicators across the world. I used the Pandas method interpolate(method='nearest', limit=5, ….) which fills the entry with the nearest existing data for

that indicator for that country, with a limit of 5 years in either direction. Interestingly, this method was unsuccessful in imputing a value for the vast majority of the nulls. This is likely because the nearby years are also null for that indicator for that country and therefore do not provide any imputable value.

3.5.2 Final Approach: Imputing by Median Values for Region Group

After imputing by nearby years proved ineffective, I implemented a standard imputation method in social science research, which is imputation by median values by subgroup. A missing value is filled with the median value for the respective region. The median is a robust form of center and therefore less sensitive to outliers.

3.6 Winsorizing the Data

To understand how many outliers the working data had, I calculated the number of outliers for each indicator. Here, I defined an outlier as a datapoint over three standard deviations away from the mean (World Bank Group, 2024). With this definition, 159 of the 218 numeric features had at least one outlier. Manually inspecting and fact-checking some of these indicators and countries with the outliers suggested that these values are not errors; they just reflect extreme or unusual circumstances in that particular country. Because of this, it is important to not simply remove these values. Instead, I used a technique called winsorization, which caps extreme values at a certain threshold rather than removing them. This is common for machine learning. For heavily skewed features, I clipped the values at the lower and upper percentiles (0.01 and 0.99).

3.7 Scaling the Data

Because I will be using k-fold cross-validation for training and testing machine learning models, scaling the data needs to happen within each fold. Scaling all the data upfront is a form of data leaking, because each scaled value is affected by all the available data and therefore will contain some information about the validation data (once that is selected).

In each machine learning model, I use SciKit Learn’s RobustScaler() method on the numeric features in the relevant fold. I selected RobustScaler() because it is less sensitive to outliers. This method transforms each feature by subtracting the median and dividing by the interquartile range (IQR), centering the data around zero while being robust to extreme values. To ensure proper cross-validation without data leakage, I implemented the scaler within a scikit-learn Pipeline. The pipeline combines a ColumnTransformer (which applies RobustScaler to numeric features while passing through one-hot encoded categorical variables unchanged) with the machine learning estimator. During cross-validation, the scaler is automatically fit only on the training folds and then applied to transform the validation fold, ensuring that no information from the validation data influences the scaling parameters. This pipeline approach guarantees that all preprocessing steps are properly contained within each fold's training process.

4. Random Forest Regression

Random Forest Regression is a good starting point for this data problem due to our continuous target variable, large number of features, and relatively small dataset (number of rows/countries). This section will describe the several different models and adjustments made to improve the Random Forest Regression results. I trained a baseline model on all the selected features, then hypertuned the parameters. After that, I trained a baseline model on the reduced number of features, and then hypertuned the parameters.

4.1 Complete Feature Set

4.1.1 Random Forest Baseline Model

Parameters: n_estimators = 500, max_depth=None

This model had a cross-validated RMSE mean of 6.6987, and a cross-validated R^2 mean of 0.8839. Even as a first attempt, this is a well-performing model. The metric of interest is Root Mean Squared Error (RMSE), but I will also be calculating R^2 for reference. My next step is to hypertune the parameters to try to increase performance (reduce RMSE).

4.1.2 Random Forest Tuned Model

Parameters after tuning: n_estimators=100, max_depth=5

I used GridSearchCV with the parameter grid 100, 250, and 500 for n_estimators, and None, 5, and 10 for max_depth. GridSearchCV performs the cross validation on each combination of the number of estimators and the max depth, and optimizes to minimize the root mean squared error. In this case, this means the scoring='neg_mean_squared_error'. After doing this, the best parameters are found to be n_estimators=100, and max_depth=5.

This tuned model had a cross-validated RMSE of 6.5490, and a cross-validated R^2 mean of 0.8883. This is a minor increase in performance from the baseline model: RMSE mean decreased by 0.2, and R^2 increased by 0.0044 from the non-tuned model.

4.1.3 Feature Importance

Using the baseline random forest regressor on the complete selected feature set provided the following most important features:

- Adjusted savings: particulate emission damage (% of GNI)
- Current health expenditure per capita, PPP (current international $)
- Domestic general government health expenditure per capita, PPP (current international $)
- Domestic general government health expenditure per capita (current US$)
- Domestic private health expenditure per capita, PPP (current international $)

These “most important features” seemed slightly unhelpful. Aside from the top feature, the next four all seemed to be referring to the same overall indicator of health expenditure. The top feature also poses the question of whether there is a different, more interpretable and explainable feature that is highly correlated with this one. In current literature, “there is no consensus on the interpretation of the importance measures when the predictors are correlated and more precisely there is no consensus on what is the effect of this correlation on the importance measures” (Gregorutti et al., 2016). Because of this, I decided to try the random forest models on the reduced feature set, which grouped highly correlated features and selected one representative feature from each group.

4.2 Reduced Feature Set

4.2.1 Random Forest Baseline Model

Parameters: n_estimators = 500, max_depth=None

This model had a cross-validated RMSE mean of 8.0737, and a cross-validated R^2 mean of 0.8328. This is worse performance than both the baseline and tuned models on the full dataset. The RMSE increased by over 20%. Again, my next step was to hypertune the parameters to try to increase performance.

4.2.2 Random Forest Tuned Model

Parameters after tuning: n_estimators=500, max_depth=10

The GridSearchCV was carried out in a similar manner to Section 4.1.2 and resulted in performance increase from the random forest baseline model on the reduced features, but did not become a top performing model. A table of all models, scores, and run times can be found in the Appendix (9.3). Random forest had a very good run time for its level of accuracy.

4.2.3 Feature Importance

Using the baseline random forest regressor on the reduced feature set provided the following most important features:

- Life expectancy at birth, total (years)
- Adjusted net national income per capita (current US$)
- Domestic private health expenditure per capita (current US$)
- Vulnerable employment, total (% of total employment) (modeled ILO estimate)
- Mortality from CVD, cancer, diabetes or CRD between exact ages 30 and 70, female (%)

These results seem to provide more explainable insight than the feature importance from the non-reduced dataset. Life expectancy at birth, total (years) is the top most important feature for the random forest regressor after removing highly correlated features.

5. Multivariable Linear Regression

Linear Regression is a simpler model than the Random Forest models developed above. It is generally more interpretable, but sometimes struggles with highly correlated features or nonlinear relationships. Models were tested on both the complete feature set and the reduced correlation-grouped feature set.

5.1 Complete Feature Set

5.1.1 Linear Regression Baseline Model

The baseline linear regression model was trained using all selected features without any regularization. This resulted in a cross-validated RMSE mean of 75.2241 and a cross-validated R^2 mean of –72.2464. These results are significantly worse than the random forest models. The negative R^2 mean displays that the model predicts the target variable considerably worse than the mean. This is expected due to the nature of the dataset, but good for comparison nevertheless.

5.1.2 Ridge Regression Model

Parameters: alpha=1.0

Ridge regression adds an L_2 penalty that shrinks coefficients. Ridge is helpful when features are highly correlated, as they are with this dataset. With default regularization strength (alpha=1.0), the model improved substantially compared to baseline linear regression. The cross-validated RMSE mean was 52.5970 and the cross-validated R^2 mean was -32.3169. The model performance is still significantly worse than the random forest models, but the RMSE decreased by approximately 30% from the non-regularized linear regression performance, indicating that regularization is essential for this dataset.

The next step is to hypertune the regularization parameter.

5.1.3 Ridge Regression Tuned Model

Parameters after tuning: alpha=5.2983169

I performed a search over a log-spaced range of alpha values (0.001 to 1000) using 5-fold cross-validation with RMSE as the scoring metric. The best performing alpha value was approximately 5.3. The tuned model achieved a cross-validated RMSE mean of 24.8444 and a cross-validated R^2 mean of -4.3423. This is a dramatic improvement over the untuned Ridge model: the RMSE decreased by over 50%. However, this model is still far less accurate than random forest.

5.2 Reduced Feature Set

5.2.1 Baseline Linear Regression Model (Reduced Features)

After grouping highly correlated features and selecting one representative from each group, the baseline linear regression model performed significantly better. The cross-validated RMSE mean was 23.2418, and the cross-validated R^2 mean was -0.9061. Compared to the baseline model on the full dataset, RMSE decreased from 75.22 to 23.24. While this model still does not have useful predictive power (it maintains a negative R^2), it clearly demonstrates the effect of reducing highly correlated features when working on a linear regression model with multicolinearity.

6. Neural Network

Neural networks are also explored as a potential machine learning model. Due to the relatively small number of observations (countries), but large number of features, neural networks are at risk of overfitting. Below, I follow the machine learning workflow of designing a baseline model, designing a model that intentionally overfits, and trying to make it less complex until it predicts with better accuracy. I used Keras from Tensorflow for the following. All of the below used adam as the optimizer, due to its efficiency and minimal need for hyperparameter tuning. Relu was used as the activation function for hidden layers, and the output layer has a linear activation function because this is a regression problem aiming to predict arbitrary values rather than categories. The loss function is mean squared error (mse).

6.1 Complete Feature Set

6.1.1 Baseline Neural Network

Architecture:

- Dense(32, relu)
- Dense(1, linear)

Training Parameters:

- epochs = 100
- batch_size = 32

This is a baseline model to establish performance before adjusting architecture and regularization. The cross-validated RMSE mean is 29.8374, which is worse than the random forest models.

6.1.2 Large Neural Network (Purposefully Overfitted)

Architecture:

- Dense(256, relu)
- Dense(128, relu)
- Dense(64, relu)
- Dense(1, linear)

Training Parameters:

- epochs = 100
- batch_size = 32

This is meant to be the “scaled up” model that intentionally overfits. This model performed worse than the baseline, with a cross-validated RMSE mean of 33.4756.

6.1.3 Regularized Neural Network

To combat overfitting, I introduce L2 weight regularization and dropout, which may help with a small dataset.

Architecture:

- Dense(256, relu, L2=0.001)
- Dropout(0.5)
- Dense(128, relu, L2=0.001)
- Dropout(0.5)
- Dense(64, relu)
- Dense(1)

Training Parameters:

- epochs = 100
- batch_size = 32
- λ (L2 penalty) = 0.001
- dropout = 0.5

This model results in a cross-validated RMSE mean of 12.6186, which is a major improvement over the previous neural networks. This further indicates that strong regularization is needed.

6.1.4 Tuned Regularized Neural Network

Architecture:

- Dense(256, relu, L2=0.001)
- Dropout(0.4)
- Dense(128, relu, L2=0.001)
- Dropout(0.4)
- Dense(1, linear)

Training Parameters:

- epochs = 150
- batch_size = 8
- $\lambda = 0.001$
- dropout_rate = 0.4

The model achieves a cross-validated RMSE mean of 10.7842. This is the best RMSE out of all neural network models that I tried, but it still does not outperform random forest.

6.1.5 Other Adjustments

In addition to the above versions of the model, I also tried alternate architectures. I tried a wide and shallow network, which was outperformed by a deep and narrow network. A complete table of the architectures, parameters, and performances is available in the appendix.

6.2 Reduced Feature Set

6.2.1 Regularized Neural Network (Reduced Features)

I trained a regularized model on the reduced feature set (one representative feature per correlation group). The architecture and training parameters can be found in the appendix. The cross-validated RMSE mean was 12.0396, which is similar to the regularized model on the full feature set.

7. Model Comparison

Random Forest was the best performing model for predicting poverty rates on World Bank Data. The table below shows the variation with the highest performance for each category of machine learning model (Random Forest, Linear Regression, and Neural Network).

Best Performing Model Version for Each Type of Model	**CV RMSE Mean**
Random Forest: Hypertuned, Full Feature Set	6.547
Linear Regression: Baseline, Reduced Feature Set	23.242
Neural Network: Hypertuned, Regularized, Full Feature Set	10.784

Figure 5: RMSE Table for the Best Performing Model Versions of Each Type of Model

This aligns with what we know about the dataset. It has a lot of features with relatively few observations, many highly correlated features, and possible non-linear relationships. Because of this, Random Forest seems to be the best choice for a model and our RMSE results confirm this. Furthermore, the hypertuned, full feature set random forest model achieved an R^2 of 0.886.

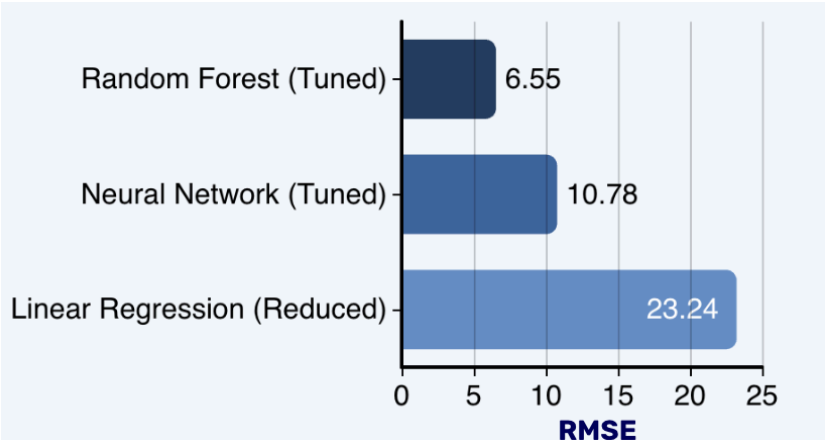
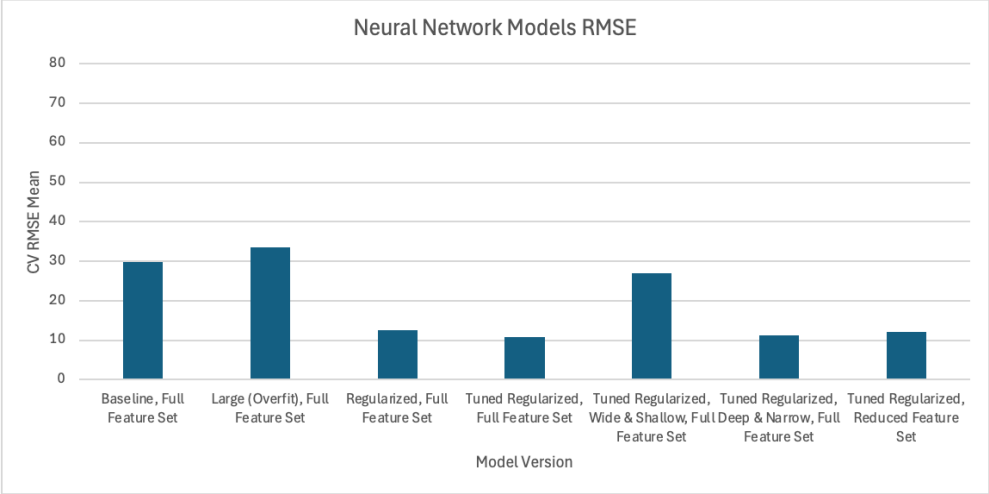
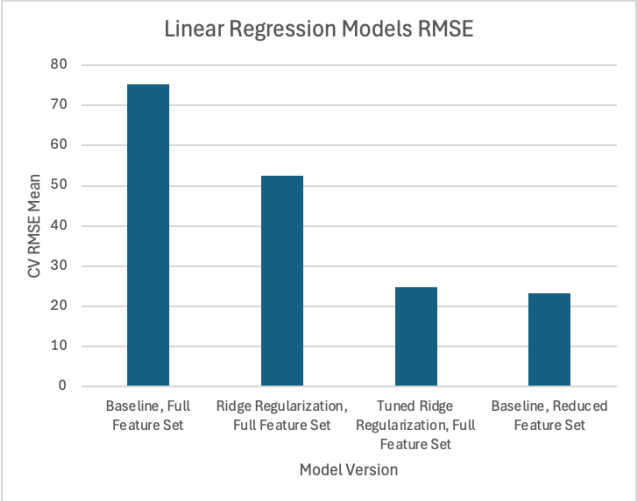


Figure 6: RMSE Graph for the Best Performing Model Versions of Each Type of Model





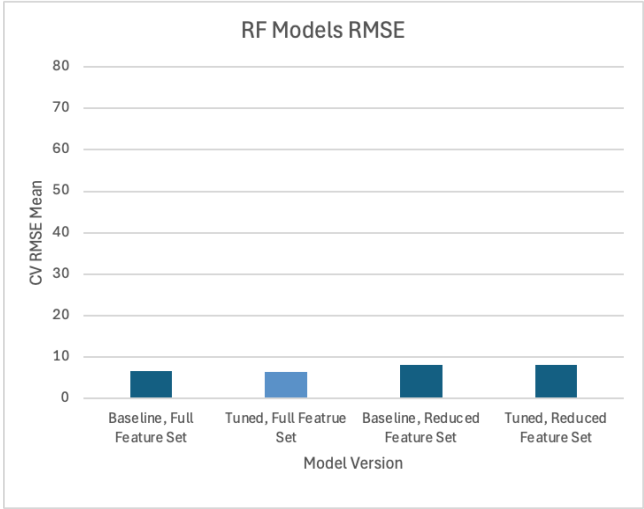


Figure 7, 8, 9: The above 3 graphs show the performance of all explored model versions, with consistent Y-axis scales for comparison

8. Conclusion

In conclusion, poverty rates in this dataset could be most effectively predicted with a tuned random forest model on the unreduced set of selected features. This model achieved a root mean squared error of 6.547 and an R^2 value of 0.888. This is significantly better than random prediction, indicating that our model has statistical power and has far less overfitting than the other models explored.

Another major takeaway is the importance of regularization for linear regression and neural networks. Dramatic improvements were found when implementing Ridge regularization for linear regression and implementing dropout/L2 for neural networks. Even though these models do not outperform random forest models for this task, it remains a good demonstration of the power of regularization. Likewise, reducing highly correlated features improved performance significantly on linear regression tasks, confirming that linear regression performs relatively poorly with multicollinearity. Furthermore, the reduced feature set led to more interpretable top features when exploring random forest feature regression. For this problem, life expectancy at birth was found to be the most important feature in predicting poverty rates.

9. Appendix

9.1 Columns of Each CSV File

Series

- Series Code
- Topic
- Indicator Name
- Short definition
- Long definition
- Unit of measure
- Periodicity
- Base Period
- Other notes
- Aggregation method
- Limitations and exceptions
- Notes from original source
- General comments
- Source
- Statistical concept and methodology
- Development relevance
- Related source links
- Other web links
- Related indicators
- License Type

Country Series

- CountryCode
- SeriesCode
- DESCRIPTION

Countries

- Country Code
- Short Name
- Table Name
- Long Name
- 2-alpha code
- Currency Unit
- Special Notes
- Region
- Income Group
- WB-2 code
- National accounts base year
- National accounts reference year

- SNA price valuation
- Lending category
- Other groups
- System of National Accounts
- Alternative conversion factor
- PPP survey year
- Balance of Payments Manual in use
- External debt Reporting status
- System of trade
- Government Accounting concept
- IMF data dissemination standard
- Latest population census
- Latest household survey
- Source of most recent Income and expenditure data
- Vital registration complete
- Latest agricultural census
- Latest industrial data
- Latest trade data
- Latest water withdrawal data

Footnotes
- CountryCode
- SeriesCode
- Year
- DESCRIPTION

main_data
- 'Country Name', 'Country Code', 'Indicator Name', 'Indicator Code',
'1960', '1961', '1962', '1963', '1964', '1965', '1966', '1967', '1968', '1969', '1970', '1971', '1972', '1973', '1974', '1975', '1976', '1977', '1978', '1979', '1980', '1981', '1982', '1983', '1984', '1985', '1986', '1987', '1988', '1989', '1990', '1991', '1992', '1993', '1994', '1995', '1996', '1997', '1998', '1999', '2000', '2001', '2002', '2003', '2004', '2005', '2006', '2007', '2008', '2009', '2010', '2011', '2012', '2013', '2014', '2015', '2016', '2017', '2018', '2019', '2020', '2021', '2022', '2023'

series_time
- SeriesCode
- Year
- DESCRIPTION

9.2 Selected Topics
- Economic Policy & Debt: National accounts: US$ at current prices: Expenditure on GDP
- Economic Policy & Debt: National accounts: Local currency at current prices: Expenditure on GDP
- Economic Policy & Debt: National accounts: US$ at constant 2015 prices: Expenditure on GDP
- Economic Policy & Debt: National accounts: Growth rates
- Economic Policy & Debt: National accounts: Local currency at constant prices: Expenditure on GDP
- Economic Policy & Debt: National accounts: Shares of GDP & other
- Economic Policy & Debt: Purchasing power parity
- Economic Policy & Debt: National accounts: US$ at constant 2015 prices: Value added

- Economic Policy & Debt: National accounts: US$ at current prices: Value added
- Economic Policy & Debt: National accounts: Local currency at current prices: Value added
- Economic Policy & Debt: National accounts: Local currency at constant prices: Value added
- Economic Policy & Debt: National accounts: Adjusted savings & income
- Economic Policy & Debt: National accounts: Local currency at constant prices: Other items
- Environment: Natural resources contribution to GDP
- Economic Policy & Debt: National accounts: US$ at current prices: Aggregate indicators
- Economic Policy & Debt: National accounts: Local currency at current prices: Aggregate indicators
- Economic Policy & Debt: National accounts: US$ at constant 2015 prices: Aggregate indicators
- Economic Policy & Debt: National accounts: Local currency at constant prices: Aggregate indicators
- Economic Policy & Debt: National accounts: Atlas GNI & GNI per capita
- Social Protection & Labor: Performance
- Education: Outcomes
- Education: Participation
- Education: Inputs
- Education: Efficiency
- Health: Mortality
- Health: Nutrition
- Health: Health systems
- Poverty: Income distribution
- Social Protection & Labor: Economic activity
- Social Protection & Labor: Labor force structure
- Social Protection & Labor: Unemployment
- Public Sector: Conflict & fragility

9.3 Model Performance and Run Time

Model	CV R^2 mean	CV RMSE mean	Run Time
Random Forest (basic, full feature set) n_estimators = 500, max_depth=None, n_jobs=-1	0.8842204529326754	6.698722333544973	5.2s
Random Forest (tuned parameters, full feature set) n_estimators=100, max_depth=5, scoring='neg_mean_squared_error'	0.8884228917957172	6.547409525073675	12.7s
Random Forest (basic, reduced feature set) n_estimators = 500, max_depth=None, n_jobs=-1	0.8328136466841143	8.073689767083371	4s
Random Forest (tuned parameters, reduced feature set) n_estimators=500, max_depth=10	0.8858460252178139	8.062280711724188	22.1s

scoring=’neg_mean_squared_error’			
Linear Regression (basic, full feature set)	-72.24639577808139	75.22414350303802	0.3s
Linear Regression (Ridge regularization, full feature set) alpha=1.0	-32.31691337047751	52.597004897440016	1.4s
Linear Regression (Ridge with tuned parameters, full feature set) alpha=5.2983169	-4.342306596858576	24.844359109744012	46.6s
Linear regression (basic, reduced feature set)	-0.9060701424370172	23.241758581409524	0.1s
Neural network (baseline model, full feature set) layers.Dense(32, activation=’relu’, layers.Dense(1, activation=’linear’ epochs=100, batch_size=32	N/A	29.8374	15.0s
Neural network (large model, full feature set) layers.Dense(256, activation='relu'), layers.Dense(128, activation='relu'), layers.Dense(64, activation='relu'), layers.Dense(1, activation='linear') epochs=100, batch_size=32	N/A	33.4756	16.5s
Neural network (regularized model, full feature set) # L2 Regularization on the kernel weights (Lambda=0.001) layers.Dense(256, activation='relu', kernel_regularizer=regularizers.l2(0.001)), # Dropout to randomly switch off 50% of neurons layers.Dropout(0.5), layers.Dense(128, activation='relu', kernel_regularizer=regularizers.l2(0.001)), layers.Dropout(0.5), layers.Dense(64, activation='relu'), layers.Dense(1, activation='linear') epochs=100, batch_size=32, lambda=0.001, dropout=0.5	N/A	12.6186	18.1s

Neural network (tuned regularized model, full feature set)
# Hidden Layer 1: Apply L2 regularization and Dropout 	
layers.Dense(256, activation='relu', kernel_regularizer=regularizers.l2(l2_lambda)),
layers.Dropout(dropout_rate),
# Hidden Layer 2: Apply L2 regularization and Dropout 	
layers.Dense(128, activation='relu', kernel_regularizer=regularizers.l2(l2_lambda)),
layers.Dropout(dropout_rate),
# Output Layer 	
layers.Dense(1, activation='linear')
 	
epochs=150,
batch_size=8,
lambda=0.001, dropout_rate=0.4
	N/A	10.7842	1m 26.5s

Neural network (tuned regularized wide and shallow model, full feature set) layers.Dense(512, activation='relu', kernel_regularizer=regularizers.l2(l2)), # Wider first layer layers.Dropout(dropout), layers.Dense(1, activation='linear') epochs=150, batch_size=8, lambda=0.001, dropout_rate=0.4	N/A	26.9264	10.4s
Neural network (tuned regularized deep and narrow model, full feature set) layers.Dense(128, activation='relu', kernel_regularizer=regularizers.l2(l2)),layers.Dropout(dropout), layers.Dense(64, activation='relu', kernel_regularizer=regularizers.l2(l2)), layers.Dropout(dropout), layers.Dense(32, activation='relu', kernel_regularizer=regularizers.l2(l2)), # Added another narrow layer layers.Dropout(dropout), layers.Dense(1, activation='linear') epochs=150, batch_size=8, lambda=0.001, dropout_rate=0.4	N/A	11.2951	9.3s
Neural network (tuned regularized, reduced feature set)	N/A	12.0396	1m

layers.Dense(32, activation='relu', kernel_regularizer=regularizers.l2(l2_lambda), layers.Dropout(dropout_rate), layers.Dense(16, activation='relu', kernel_regularizer=regularizers.l2(l2_lambda)), layers.Dropout(dropout_rate), layers.Dense(1) epochs=150, batch_size=8, lambda=0.001, dropout_rate=0.2			

10. References

References

Gregorutti, B., Michel, B., & Saint-Pierre, P. (2016, March 23). *Correlation and variable importance in random forests*. Springer Nature Link. Retrieved December 1, 2025, from https://link.springer.com/article/10.1007/s11222-016-9646-1

Handling "Missing Data" Like a Pro - Part 2- Imputation Methods. (2021, July 9). Towards Data Science. Retrieved December 1, 2025, from https://towardsdatascience.com/handling-missing-data-like-a-pro-part-2-imputation-methods-eabbf10b9ce4/

Outlier detection and treatment. (n.d.). The World Bank. Retrieved December 1, 2025, from https://thedocs.worldbank.org/en/doc/20f02031de132cc3d76b91b5ed8737d0-0050012017/related/lecture-12-1.pdf

World Bank Group. (2024). *World Bank's Global Data*. Kaggle. https://www.kaggle.com/datasets/vijayveersingh/world-banks-global-indicator-data

Yang, S., Yuan, L., Lai, Y. C., Shen, X., Wonka, P., & Ye, J. (2012). Feature Grouping and Selection Over an Undirected Graph. KDD : proceedings. International Conference on Knowledge Discovery & Data Mining, 922–930. https://doi.org/10.1145/2339530.2339675